LevelUp

Software Dependability Report

Pierpaolo Cammardella
Matricola: NF22500011

Università degli Studi di Salerno
Dipartimento di Informatica
August 31, 2026

Contents

1 Introduction

1.1 Project Overview

LevelUp is a Java web application originally developed in 2021 for the TSW exam as a e-commerce platform to sell online courses built with JSP, Servlets, and Apache Tomcat, backed by a MySQL database. For this exam, the project was revived from a non-functional state, made buildable and testable again, and used as the target for applying the dependability techniques covered by the course: continuous integration, formal specification, containerization, automated testing, coverage analysis, mutation testing, performance benchmarking, and security analysis.

1.2 Architecture

The application follows a layered, MVC structure

Controllers are implemented as HTTP Servlets (`controller` package), one per resource area (`HomeServlet`, `UserServlet`, `ManagerServlet`, `AccountServlet`, `CourseServlet`). Each servlet maps multiple logical actions under a single URL prefix (e.g. `/manager/*`), dispatching internally on `request.getPathInfo()` via a switch statement rather than one servlet per action. Request validation is handled by a small validator hierarchy (`RequestValidator` and its subclasses), which accumulate field-level errors before a controller proceeds.

The model/persistence layer follows a Manager–DAO–Query pattern: each domain entity (`Utente`, `Corso`, `Carrello`, `Ordine`, `Categoria`, `Tag`, `Oggetto`) has a corresponding `Manager` class implementing a `Dao` interface, which delegates SQL construction to a dedicated `Query` class. Managers obtain connections from a shared connection pool (`ConPool`, backed by Tomcat JDBC) and execute queries against MySQL via plain JDBC.

Views are JSP pages under `WEB-INF/views`, protected from direct access and reachable only via servlet forwarding, organized by feature area (e.g. `logging/`, `admin/`, `user/`, `home/`). Session state (`HttpSession`) carries the logged-in user and their shopping cart across requests, independent of the persistence layer until checkout or logout explicitly synchronizes them to the database.

1.3 Repository Information

- GitHub repository: https://github.com/Pierbit/LevelUpProjectSD
- Docker Hub image: https://hub.docker.com/r/pierbit/levelup-app

2 Build and Continuous Integration

2.1 Local Build

The project builds via Maven (`mvn clean package`), producing a deployable `.war` artifact for Apache Tomcat 9.

2.2 CI/CD Pipeline

A GitHub Actions workflow (`.github/workflows/build.yml`) runs automatically on every push. It:

- Checks out the repository and sets up JDK 16.
- Spins up a MySQL service container, initialized with the project's schema and seed data (`db-init/schema.sql`, `db-init/seed.sql`), so the full integration test suite can run against a real database, not just an isolated compile step.
- Runs `mvn clean package`, which compiles the project and executes the full JUnit test suite.

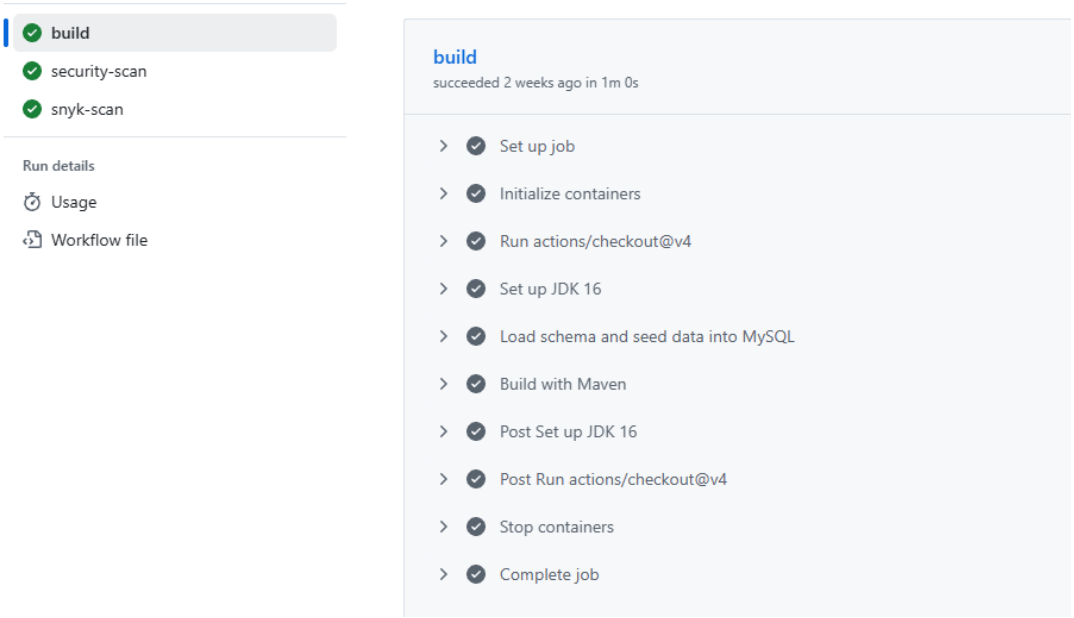


Figure 1: GitHub Actions CI/CD build

3 Formal Specification with JML / OpenJML

A set of core methods spanning pagination, request validation, cart management, pricing, and user registration were annotated with JML preconditions, postconditions, and class invariants, and verified using OpenJML's extended static checking (`-esc`) mode.

Verified methods include:

- `Paginator` (all methods)
- `RequestValidator.required()` and `gatherError()`
- `Carrello.addOggetto()`, `Oggetto.setPrezzo()`
- `RegisterValidator.isValidUsername()` / `isValidPassword()`

```
public class Paginator {  30 usages  Pierbit
    //@ invariant limit > 0;
    //@ invariant offset >= 0;

    private /*@ spec_public @*/ final int limit;  4 usages
    private /*@ spec_public @*/ final int offset;  2 usages

    //@ requires page >= 1 && page <= 1000000;
    //@ requires itemsPerPage > 0 && itemsPerPage <= 1000;
    //@ ensures limit == itemsPerPage;
    //@ ensures offset == (page - 1) * itemsPerPage;
    public Paginator(int page, int itemsPerPage) {  14 usages  Pierbit
        this.limit = itemsPerPage;
        this.offset = (page == 1) ? 0 : ((page - 1) * itemsPerPage);
    }

    //@ ensures \result == limit;
    public int getLimit() { return limit; }

    //@ ensures \result == offset;
    public int getOffset() { return offset; }

    //Per sapere quante pagine creare (quanti record presenti)
    //Se c'è resto serve una pagina aggiuntiva
    //@ requires size >= 0;
    //@ ensures \result == (size / limit) + (size % limit == 0 ? 0 : 1);
    public int getPages(int size){  11 usages  Pierbit
        int additionalPage = (size % limit==0) ? 0 : 1;
        return (size/limit) + additionalPage;
    }
}
```

Figure 2: JML specification of Paginator class

4 Docker Containerization and Orchestration

4.1 Image and Orchestration

A multi-stage `Dockerfile` builds the application (Maven build stage, then a Tomcat runtime stage) and a `docker-compose.yml` orchestrates the application alongside a MySQL container, auto-initialized from `schema.sql` and `seed.sql`. The image is published to Docker Hub.

4.2 Issues identified by Containerization

Moving the application into a Linux-based container exposed a latent defect that had been silently masked by the original Windows development environment:

- A table-name case mismatch (`utenteCreacorso` vs. `utenteCreaCorso`) in `CorsoQuery.getUtenteCreatore()`, invisible on Windows/MySQL's case-insensitive default, but a runtime SQL error on Linux.

The issue was fixed once identified, this represents a concrete example of containerization acting as fault-avoidance mechanism.

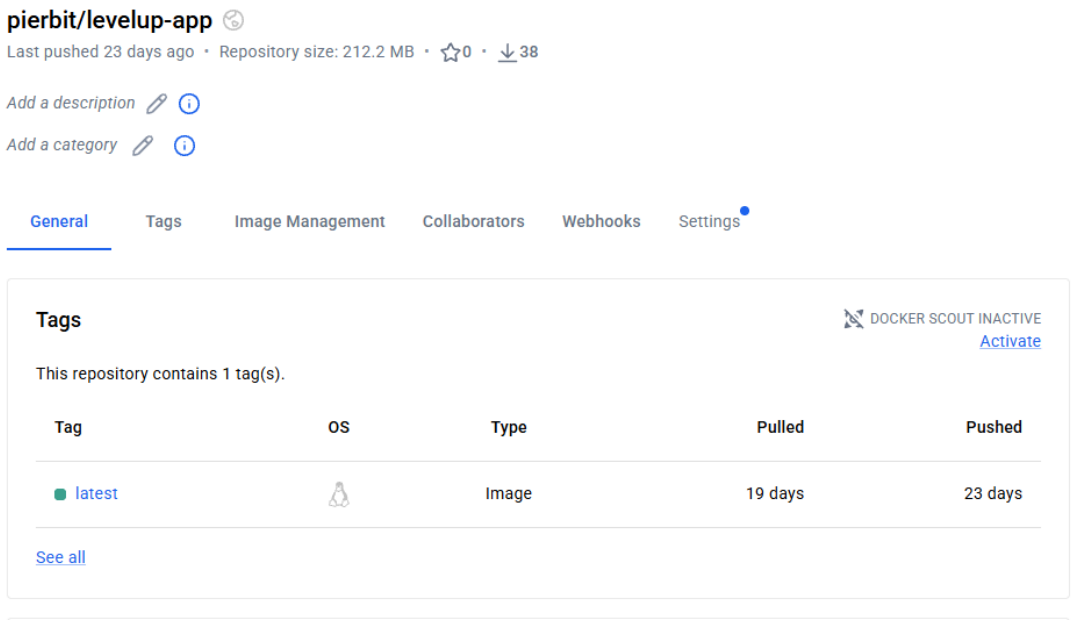


Figure 3: Docker Hub image

```
CONTAINER ID   IMAGE                  COMMAND                  CREATED              STATUS              PORTS
                          NAMES
efdb648dacb7   levelupprojectsd-app   "catalina.sh run"        About a minute ago   Up About a minute   0.0.0.0:8080->80
80/tcp, [::]:8080->8080/tcp   levelupprojectsd-app-1
fbfd00320494   mysql:8.0              "docker-entrypoint.s…"   2 weeks ago          Up About a minute   3306/tcp, 33060/
tcp                           levelupprojectsd-db-1
```

Figure 4: Docker Compose orchestration: both containers running

5 Automated Testing

5.1 Approach

The test suite combines two deliberately different strategies, matched to what each part of the codebase actually is:

- **Unit tests** (JUnit 5 + Mockito) for pure logic and HTTP-layer code (`Paginator`, `RequestValidator`, `RegisterValidator`, `LoginValidator`, and mocked servlet-level tests).
- **Integration tests** against a real MySQL database for the Manager/DAO layer (`UtenteManager`, `CorsoManager`, `CarrelloManager`, `OrdineManager`, `CategoriaManager`, `TagManager`, `OggettoManager`), since this layer's entire purpose is coordinating with the database correctly.

5.2 Test Generation Approach

All tests in this suite are automatically executed via `mvn test` locally and on every push through the CI/CD pipeline. but were not automatically generated. Every test scenario was manually identified and written, based on direct analysis of each class's intended behaviour.

5.3 Defects Found

Writing tests surfaced several defects in the pre-existing codebase:

1. **Search silently excludes unassociated courses.** `CorsoQuery.search()` performs an implicit inner join across four tables; a course missing a category, tag, or creator association is invisible to every search query. The standard user-facing course creation flow never sets a category or tag, meaning user-created courses are unsearchable by default.
2. **Several methods/lines of unreachable code**
3. **SQL statements that threw on every call and unimplemented stubs**

6 Code Coverage Analysis (JaCoCo)

Coverage was measured with the JaCoCo Maven plugin (`prepare-agent` + `report` goals bound to the `test` phase).

Result: 32% instruction coverage, 20% branch coverage overall.

Coverage is concentrated in the Manager/model layer (50%–88% per package), matching where the test suite deliberately focused effort, and low in the controller/servlet layer (14%–49%), which is largely thin coordination code that delegates to already-tested Manager methods.

LevelUp

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
controller		14%		11%	270	322	1,255	1,461	58	96	4	11
benchmark.jmh_generated		0%		0%	80	80	297	297	15	15	5	5
model.utente		50%		36%	49	96	158	296	30	73	0	3
model.corso		72%		58%	32	95	80	277	15	67	0	3
controller.validators		27%		37%	26	36	70	91	15	20	5	7
model.ordine		51%		34%	27	50	57	119	17	37	0	3
controller.search		49%		14%	26	36	25	50	1	10	0	3
model.carrello		58%		40%	9	24	22	55	5	19	0	3
model.categoria		73%		56%	9	31	20	80	4	23	0	3
model.oggetto		80%		61%	10	37	17	81	4	28	0	3
benchmark		0%		n/a	3	3	5	5	3	3	1	1
model.tag		88%		71%	6	28	8	75	2	21	0	3
model.storage		77%		66%	5	9	4	27	3	6	1	4
Total	7,917 of 11,746	32%	621 of 780	20%	552	847	2,018	2,914	172	418	16	52

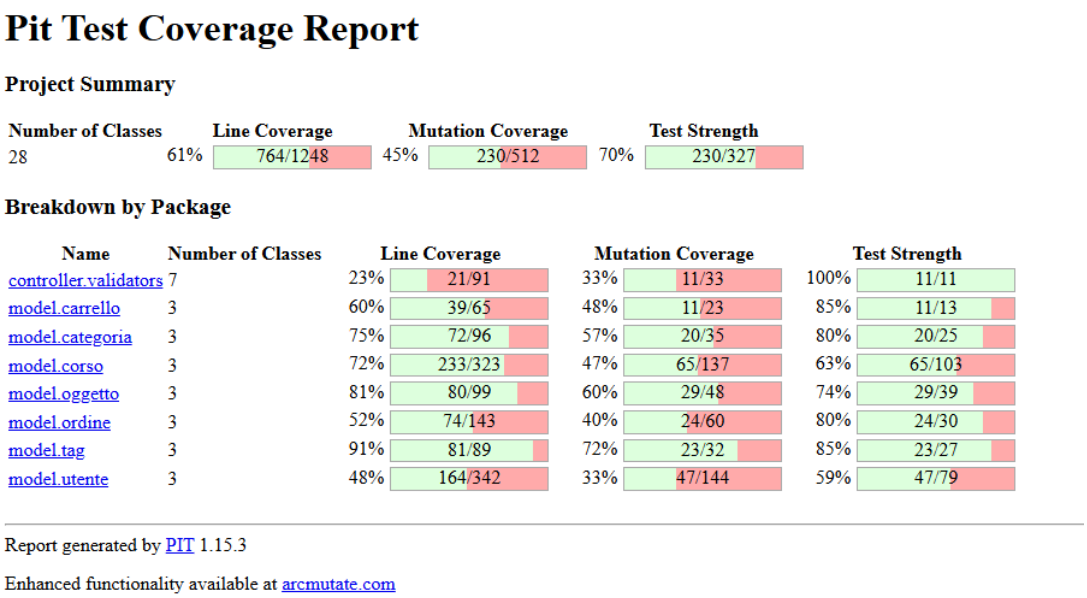
Figure 5: JaCoCo Report, package-level summary table

7 Mutation Testing (PiTest)

PiTest was configured to mutate the model and validation layer classes, run against the full JUnit test suite.

Result: 45% raw mutation score (230/512 mutants killed); 70% test strength.

The gap between the raw score and test strength is explained because 282 of 512 mutants sat on code the test suite never executed at all. Restricting the calculation to mutants that were actually exercised, the test suite killed 70% of them, indicating that where the tests are run, they reliably distinguish correct from mutated behavior.



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
28	61%	764/1248	45%	230/512	70%	230/327

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
controller.validators	7	23%	21/91	33%	11/33	100%	11/11
model.carrello	3	60%	39/65	48%	11/23	85%	11/13
model.categoria	3	75%	72/96	57%	20/35	80%	20/25
model.corso	3	72%	233/323	47%	65/137	63%	65/103
model.oggetto	3	81%	80/99	60%	29/48	74%	29/39
model.ordine	3	52%	74/143	40%	24/60	80%	24/30
model.tag	3	91%	81/89	72%	23/32	85%	23/27
model.utente	3	48%	164/342	33%	47/144	59%	47/79

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

Figure 6: PiTest report summary

8 Performance Testing (JMH)

`CorsoQuery.search()` was benchmarked with JMH (average time mode, microsecond output), as the most computationally complex pure-logic method in the codebase (string-building and branching over a list of search conditions).

Result: 0.115 ± 0.004 µs/op.

This confirms the search-condition-building logic itself is effectively free computationally. Given the application's architecture as a thin CRUD layer over MySQL, any real-world latency in the search flow originates from the database round-trip, not from in-application computation.

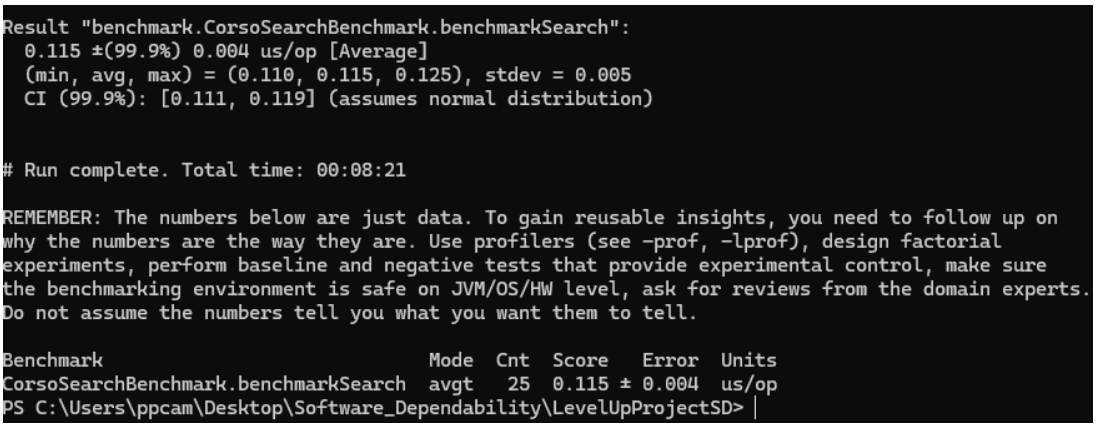


Figure 7: JMH Benchmark output table

9 Security Analysis

9.1 Tooling in CI/CD

Two security scanners run automatically as separate GitHub Actions jobs on every push: GitGuardian (secret scanning) and Snyk (dependency vulnerability scanning). SonarCloud was run against the repository for a broader static analysis (code quality, bugs, and vulnerabilities).

9.2 GitGuardian

GitGuardian identified a hardcoded MySQL root password present in `ConPool.java`, `docker-compose.yml`, and `build.yml`. As a fix, the password is now supplied exclusively via a `DB_PASSWORD` environment variable (no hardcoded fallback), sourced from a `.env` file locally/in Docker, and from a GitHub Actions secret in CI. A committed `.env.example` documents the requirement for anyone cloning the repository.

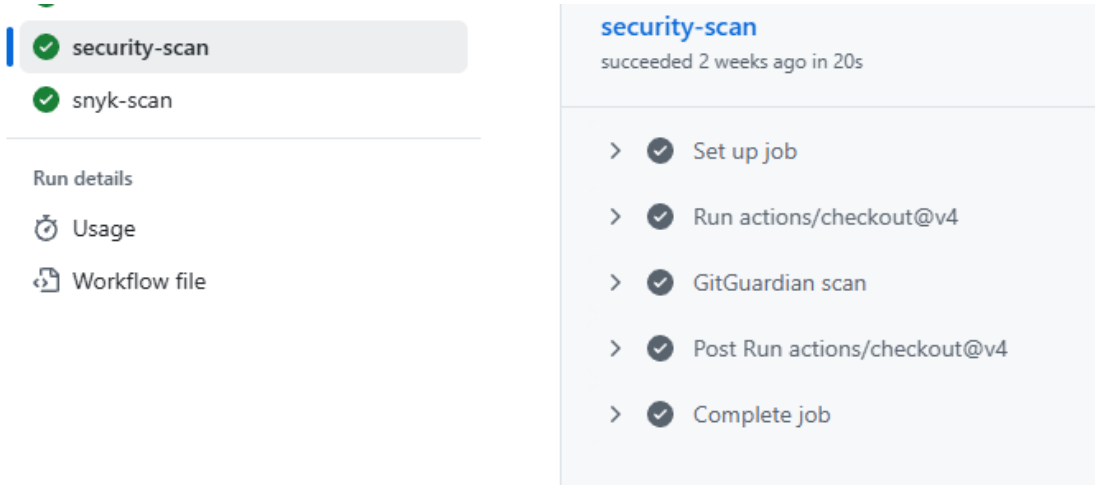


Figure 8: GitGuardian Scan output on latest commit

9.3 Snyk

Snyk identified 11 known CVEs across the project's Maven dependencies. **Fixed 10** by upgrading `mysql-connector-j` to 9.2.0, `tomcat-jdbc` to 9.0.109, and `org.json:json` to 20231013. **1 documented as an accepted risk**: an XXE (XML External Entity Injection) vulnerability in `javax.servlet:jstl:1.2`, which has no available patched version (library unmaintained), recorded via a `.snyk` policy file with an explicit justification and expiry date.

9.4 SonarCloud

SonarCloud's **Vulnerability** category surfaced two genuine issues, both remediated:

- **Weak password hashing**: `Utente.java` used SHA-1 (unsalted) for password storage. Replaced with bcrypt (`jbcrypt`), verified via a dedicated test suite covering hashing, verification, salt uniqueness, and the full login/registration flow end to end.
- **Non-serializable objects stored in `HttpSession`**: `Carrello`, `Utente`, and their nested model classes did not implement `Serializable`. Fixed by implementing `Serializable` across the affected POJO chain.

A separately flagged XML injection warning (`S6399`) was marked as a false positive in SonarCloud with documented reasoning.

Figure 9: Snyk scan output showing 0 remaining active vulnerabilities

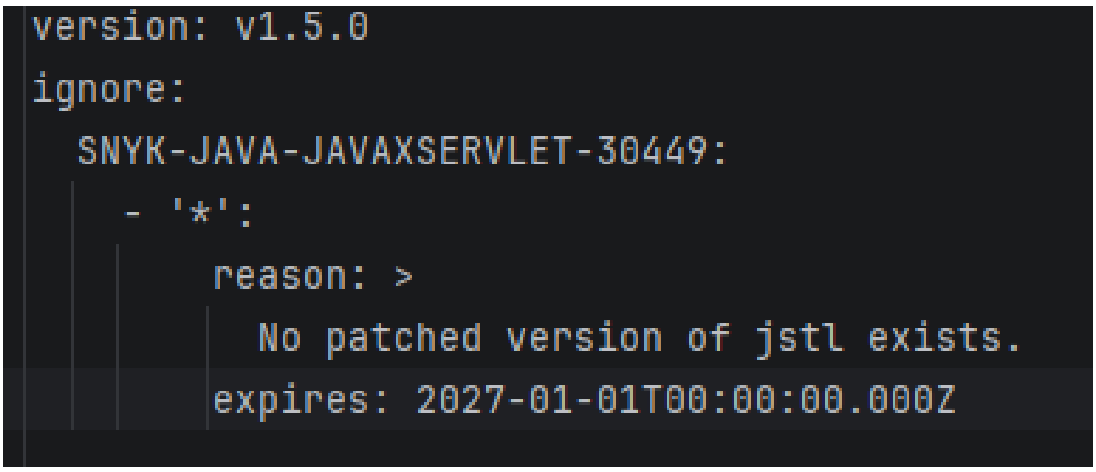


Figure 10: Snyk policy file with explicit justification

The remaining SonarCloud findings are related to **Maintainability, A-rating** (Adaptability, Intentionality, Consistency), to **Reliability. C-rating** (Intentionality, Accessibility) and **Security, B-rating** (Consistency) where the marked issues are primarily a single repeated pattern (unhandled`ServletException/IOException` from `RequestDispatcher.forward()`) across the controller layer. This pattern was fixed in `AccountServlet`, `HomeServlet`, and `ManagerServlet` as a demonstration, the remainder was scoped out as low-severity.

```java
public void setPasswordHashed(String password) {  4 usages  Pierbit
    try {
        MessageDigest digest =
                MessageDigest.getInstance( algorithm: "SHA-1");
        digest.reset();
        digest.update(password.getBytes(StandardCharsets.UTF_8));
        this.password = String.format("%040x", new
                BigInteger( signum: 1, digest.digest()));
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    }
}
```

Figure 11: Old password hashing

```java
public void setPasswordHashed(String password) {  4 usages  Pierbit *
    this.password = org.mindrot.jbcrypt.BCrypt.hashpw(password, org.mindrot.jbcrypt.BCrypt.gensalt());
}
```

Figure 12: New bcrypt password hashing

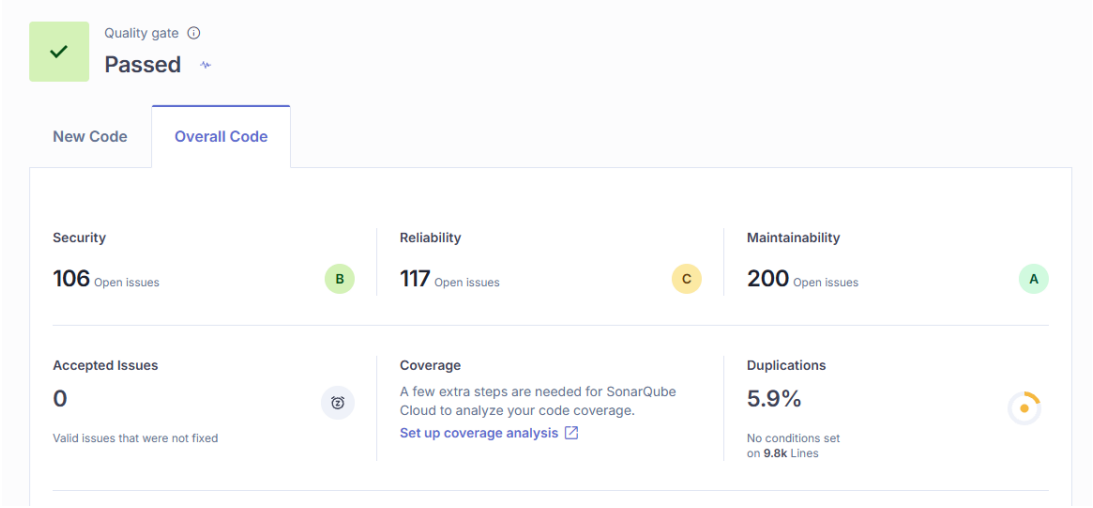


Figure 13: SonarCloud code summary overview

10 Conclusions

This project applied a full pipeline of dependability techniques to a real, previously non-functional codebase: continuous integration with automated database-backed testing, formal specification of core methods, containerized and orchestrated deployment, a substantial integration and unit test suite that surfaced multiple genuine pre-existing defects, coverage and mutation analysis to evaluate the quality of that suite, performance benchmarking, and a three-tool security pass that found and fixed real vulnerabilities. Each technique contributed to the software's correctness and robustness.